

Tugas 9.: Tugas Praktikum Mandiri

Oryza Ayunda Putri - 0110224030

Teknik Informatika, STT Terpadu Nurul Fikri, Depok
E-mail: nasi.tektekmangudin@gmail.com

1. Tugas Praktikum Mandiri

1.1 Import Library

Pada bagian awal kode, dilakukan proses import berbagai library penting yang digunakan dalam analisis dan pemodelan data. Library seperti pandas dan numpy digunakan untuk pengolahan data, termasuk manipulasi tabel dan operasi matematis. Library visualisasi seperti matplotlib.pyplot dan seaborn memungkinkan pembuatan grafik distribusi, heatmap, serta visualisasi pendukung lainnya. Selain itu, beberapa modul dari Scikit-Learn seperti train_test_split, StandardScaler, GaussianNB, accuracy_score, confusion_matrix, dan classification_report digunakan untuk pembagian data, standarisasi, pembuatan model Naive Bayes, dan evaluasi performa model. Seluruh import ini merupakan persiapan agar proses analisis dapat berjalan dengan lengkap dan terstruktur.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```

✓ 0.0s Python

1.2 Membaca Dataset

Dataset kanker payudara dibaca menggunakan fungsi `pd.read_csv()`. Dataset tersebut berisi berbagai fitur yang menggambarkan karakteristik sel, seperti radius, tekstur, perimeter, area, smoothness, dan berbagai statistik turunan lainnya. Proses pembacaan ini mengubah file CSV menjadi DataFrame yang mudah dikelola dan dianalisis. Setelah dataset berhasil dimuat, pemeriksaan awal seperti jumlah baris, kolom, dan tipe data dapat dilakukan melalui fungsi pendukung lainnya.

```
cancer_data = pd.read_csv('../data/data_cancer.csv', sep=',')
```

✓ 0.0s

1.3 Melihat Struktur Dataset

Kode `cancer_data.head()`, `cancer_data.tail()`, dan `cancer_data.info()` digunakan untuk menampilkan 5 baris pertama, 5 baris terakhir, serta informasi detail dari dataset seperti tipe data dan jumlah non-null pada setiap kolom. Tahap ini penting untuk memahami kondisi data sebelum dilakukan preprocessing. Dari hasil ini terlihat bahwa dataset memiliki satu kolom bernama Unnamed: 32 yang sepenuhnya berisi nilai kosong dan tidak memiliki fungsi penting.

cancer_data.head()

✓ 0.0s

Python

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	...	texture_worst	pe
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	...	17.33	
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	...	23.41	
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	...	25.53	
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	...	26.50	
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	...	16.67	

5 rows × 33 columns

cancer_data.info()

✓ 0.0s

Python

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 33 columns):
Column Non-Null Count Dtype

0 id 569 non-null int64
1 diagnosis 569 non-null object
2 radius_mean 569 non-null float64
3 texture_mean 569 non-null float64
4 perimeter_mean 569 non-null float64
5 area_mean 569 non-null float64
6 smoothness_mean 569 non-null float64
7 compactness_mean 569 non-null float64
8 concavity_mean 569 non-null float64
9 concave points_mean 569 non-null float64
10 symmetry_mean 569 non-null float64
11 fractal_dimension_mean 569 non-null float64
12 radius_se 569 non-null float64
13 texture_se 569 non-null float64
14 perimeter_se 569 non-null float64
15 area_se 569 non-null float64
16 smoothness_se 569 non-null float64
17 compactness_se 569 non-null float64
18 concavity_se 569 non-null float64
19 concave points_se 569 non-null float64
...
31 fractal_dimension_worst 569 non-null float64
32 Unnamed: 32 0 non-null float64
dtypes: float64(31), int64(1), object(1)
memory usage: 146.8+ KB

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

cancer_data.tail()

✓ 0.0s

Python

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	...	texture_worst	pe
564	926424	M	21.56	22.39	142.00	1479.0	0.11100	0.11590	0.24390	0.13890	...	26.40	
565	926682	M	20.13	28.25	131.20	1261.0	0.09780	0.10340	0.14400	0.09791	...	38.25	
566	926954	M	16.60	28.08	108.30	858.1	0.08455	0.10230	0.09251	0.05302	...	34.12	
567	927241	M	20.60	29.33	140.10	1265.0	0.11780	0.27700	0.35140	0.15200	...	39.42	
568	92751	B	7.76	24.54	47.92	181.0	0.05263	0.04362	0.00000	0.00000	...	30.37	

5 rows × 33 columns

1.4 Mengecek Missing Values

Tahap selanjutnya adalah memeriksa nilai kosong menggunakan perintah `cancer_data.isnull().sum()`. Hasilnya menunjukkan bahwa semua kolom memiliki nilai lengkap, kecuali kolom Unnamed: 32 yang memiliki 569 nilai kosong. Karena

kolom tersebut tidak mengandung informasi berguna dan hanya menjadi beban dalam pemrosesan, maka harus dihapus pada langkah selanjutnya.

```
cancer_data.isnull().sum()
✓ 0.0s
```

id	0
diagnosis	0
radius_mean	0
texture_mean	0
perimeter_mean	0
area_mean	0
smoothness_mean	0
compactness_mean	0
concavity_mean	0
concave points_mean	0
symmetry_mean	0
fractal_dimension_mean	0
radius_se	0
texture_se	0
perimeter_se	0
area_se	0
smoothness_se	0
compactness_se	0
concavity_se	0
concave points_se	0
symmetry_se	0
fractal_dimension_se	0
radius_worst	0
texture_worst	0
perimeter_worst	0
...	
concave points_worst	0
symmetry_worst	0
fractal_dimension_worst	0
Unnamed: 32	569
dtype: int64	

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings...](#)

1.5 Menghapus Kolom Tidak Relevan

Dua kolom dihapus dari dataset menggunakan fungsi `drop()`, yaitu `id` dan `Unnamed: 32`. Kolom `id` hanya merupakan nomor urut dan tidak memiliki kontribusi dalam proses klasifikasi. Sementara itu, `Unnamed: 32` tidak memiliki data dan hanya berisi nilai kosong, sehingga penghapusannya penting agar dataset bersih dan siap diproses lebih lanjut. Langkah ini memastikan model hanya bekerja dengan fitur-fitur yang benar-benar relevan.

```
cancer_data = cancer_data.drop(columns=['id', 'Unnamed: 32'], errors='ignore')
✓ 0.0s
```

Python

1.6 Analisis Distribusi Kelas (Class Distribution Analysis)

Kode `cancer_data['diagnosis'].value_counts()` digunakan untuk menghitung jumlah masing-masing kategori pada kolom `diagnosis` setelah dilakukan proses encoding. Pada dataset kanker payudara ini, label “M” (Malignant) telah diubah menjadi 1, sedangkan label “B” (Benign) diubah menjadi 0 sehingga keduanya dapat diproses oleh model. Hasil `value_counts()` menunjukkan berapa banyak sampel yang termasuk

kategori jinak dan berapa banyak yang termasuk kategori ganas. Informasi ini sangat penting untuk mengetahui distribusi kelas pada dataset, apakah dataset seimbang atau tidak. Pada kasus ini, jumlah sampel kanker jinak cenderung lebih banyak dibandingkan sampel kanker ganas, namun perbedaan tersebut masih dalam batas yang wajar sehingga tidak menimbulkan masalah serius pada proses pelatihan model. Evaluasi ini membantu memastikan bahwa model tidak akan terlalu condong pada salah satu kelas dan tetap bekerja dengan akurat.

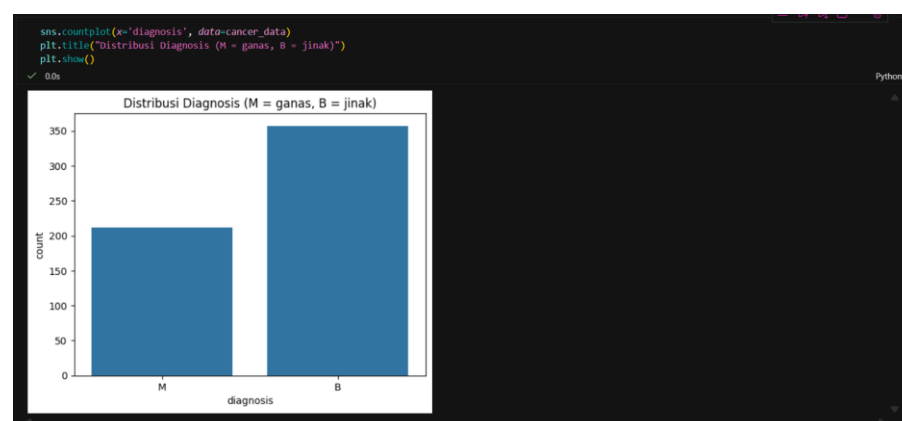
```
cancer_data['diagnosis'].value_counts()
✓ 0.0s Python
```

diagnosis	count
B	357
M	212

Name: count, dtype: int64

1.7 Visualisasi Distribusi Kelas (Class Distribution Visualization)

Kode tersebut digunakan untuk melakukan visualisasi distribusi kelas diagnosis menggunakan grafik countplot dari library Seaborn. Grafik ini menampilkan jumlah sampel untuk masing-masing kategori diagnosis, yaitu 1 (Malignant/ganas) dan 0 (Benign/jinak). Visualisasi ini sangat penting dalam analisis data karena membantu melihat apakah dataset memiliki distribusi kelas yang seimbang atau tidak. Jika salah satu kelas memiliki jumlah data jauh lebih banyak, model berpotensi bias dan cenderung memprediksi kelas tersebut. Pada kasus dataset kanker payudara, countplot menunjukkan bahwa jumlah kasus benign lebih banyak daripada malignant, namun perbedaan ini masih relatif wajar. Visualisasi seperti ini mempermudah pemahaman awal terhadap karakteristik dataset sebelum dilakukan pemodelan machine learning.



1.8 Binning / Discretization (Pembagian Data Kontinu ke dalam Kelompok Kategori)

Kode tersebut melakukan proses binning atau pengelompokan nilai numerik menjadi kategori tertentu menggunakan fungsi `pd.qcut()`. Binning dilakukan pada tiga fitur yang memiliki nilai kontinu, yaitu `radius_mean`, `texture_mean`, dan `area_mean`. Setiap fitur dibagi menjadi tiga kelompok menggunakan parameter `q=3`, yang berarti pembagian dilakukan berdasarkan tiga kuantil sehingga jumlah data dalam setiap

kategori relatif sama. Tiga label kategori yang diberikan adalah “Low”, “Medium”, dan “High”, sehingga nilai-nilai pada fitur tersebut dikonversi menjadi kategori tingkat rendah, sedang, atau tinggi. Tujuan dari proses ini adalah mempermudah interpretasi data dan memungkinkan analisis tambahan berbasis kategori. Proses binning juga berguna dalam eksplorasi data untuk memahami bagaimana fitur tertentu berhubungan dengan diagnosis kanker. Dengan mengubah data kontinu menjadi kategori, kita dapat melihat perbedaan distribusi dan pola hubungan secara lebih jelas dalam visualisasi atau analisis statistik.

```
cancer_data['radius_bin'] = pd.qcut(cancer_data['radius_mean'], q=3, labels=['Low', 'Medium', 'High'])
cancer_data['texture_bin'] = pd.qcut(cancer_data['texture_mean'], q=3, labels=['Low', 'Medium', 'High'])
cancer_data['area_bin'] = pd.qcut(cancer_data['area_mean'], q=3, labels=['Low', 'Medium', 'High'])
```

✓ 0.0s

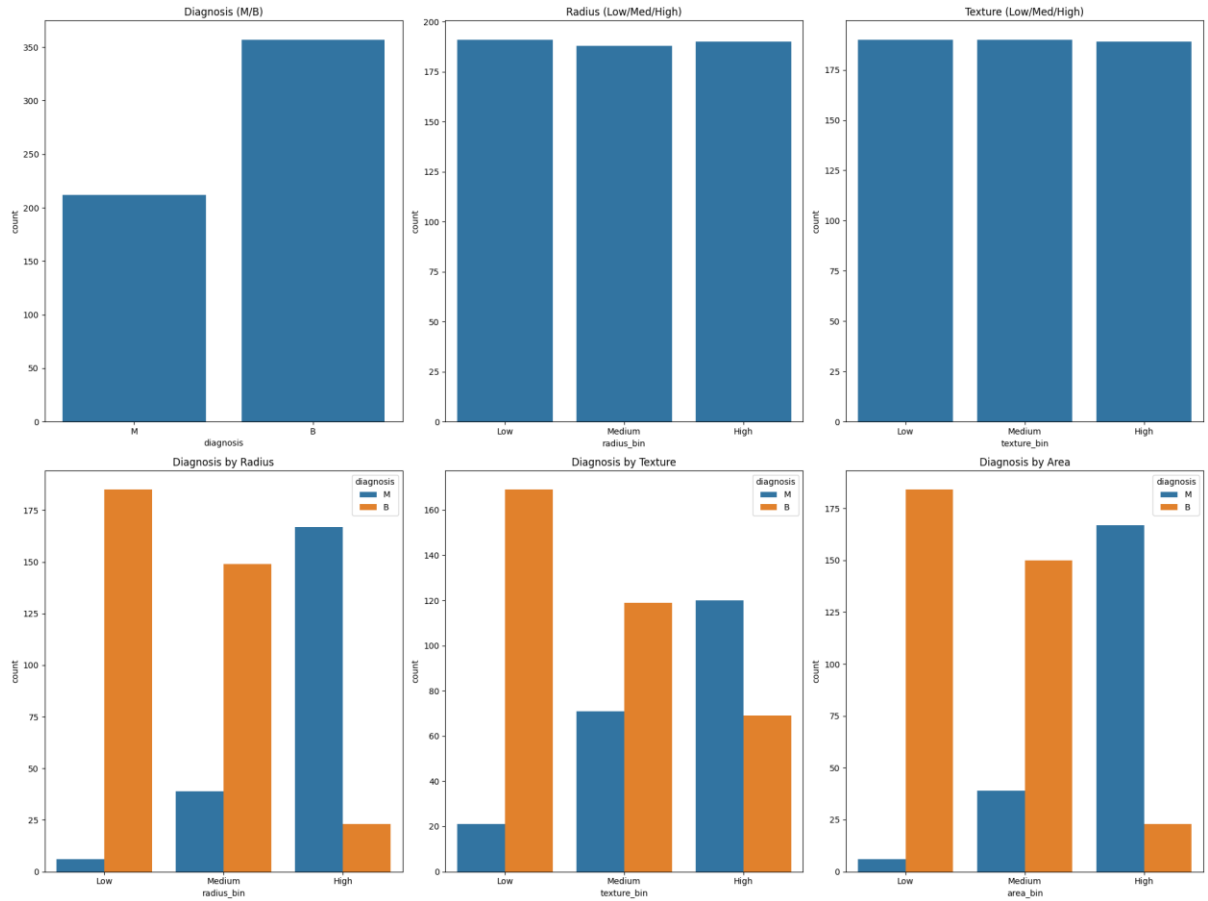
1.9 Standarisasi Fitur

Karena nilai setiap fitur memiliki skala yang berbeda-beda (misalnya area bernilai ratusan hingga ribuan, sedangkan smoothness bernilai kurang dari 1), maka dilakukan standarisasi menggunakan `StandardScaler()`. Tahap ini mengubah seluruh fitur menjadi memiliki mean 0 dan standar deviasi 1. Standarisasi sangat penting bagi Gaussian Naive Bayes karena model ini mengasumsikan bahwa data mengikuti distribusi normal. Proses standarisasi dilakukan pada data training menggunakan `fit_transform()`, sedangkan data testing hanya ditransformasi dengan `transform()` untuk menghindari kebocoran data.

1.10 Visualisasi Multiplot Distribusi Fitur dan Hubungan dengan Diagnosis

Kode tersebut digunakan untuk membuat enam visualisasi countplot secara sekaligus dalam satu kanvas menggunakan fitur subplot dari Matplotlib. Proses ini bertujuan untuk mempermudah analisis eksplorasi data (EDA) terhadap distribusi kategori pada beberapa fitur hasil binning, yaitu `radius_bin`, `texture_bin`, dan `area_bin`, serta melihat bagaimana kategori tersebut berhubungan dengan diagnosis kanker. Pada baris pertama subplot, grafik menampilkan distribusi jumlah diagnosis serta distribusi pembagian kategori pada radius dan tekstur. Pada baris kedua, grafik-grafik menampilkan hubungan antara kategori radius, tekstur, dan area dengan diagnosis kanker melalui parameter `hue='diagnosis'`. Dengan visualisasi seperti ini, pola hubungan antara fitur-fitur tersebut dengan label diagnosis dapat terlihat lebih jelas. Misalnya, kita dapat mengamati apakah kategori “High” pada radius atau area lebih banyak muncul pada diagnosis ganas (M) dibandingkan kategori lainnya. Penggunaan layout yang rapi melalui `plt.tight_layout()` memastikan bahwa setiap grafik dapat dibaca dengan jelas tanpa saling bertumpukan. Secara keseluruhan, blok kode ini membantu memahami fitur

mana yang memiliki pengaruh besar dalam membedakan kanker jinak dan kanker ganas.



```

fig, axes = plt.subplots(nrows=2, ncols=3, figsize=(20, 15))
axes = axes.flatten()

sns.countplot(x='diagnosis', data=cancer_data, ax=axes[0])
axes[0].set_title('Diagnosis (M/B)')

sns.countplot(x='radius_bin', data=cancer_data, ax=axes[1])
axes[1].set_title('Radius (Low/Med/High)')

sns.countplot(x='texture_bin', data=cancer_data, ax=axes[2])
axes[2].set_title('Texture (Low/Med/High)')

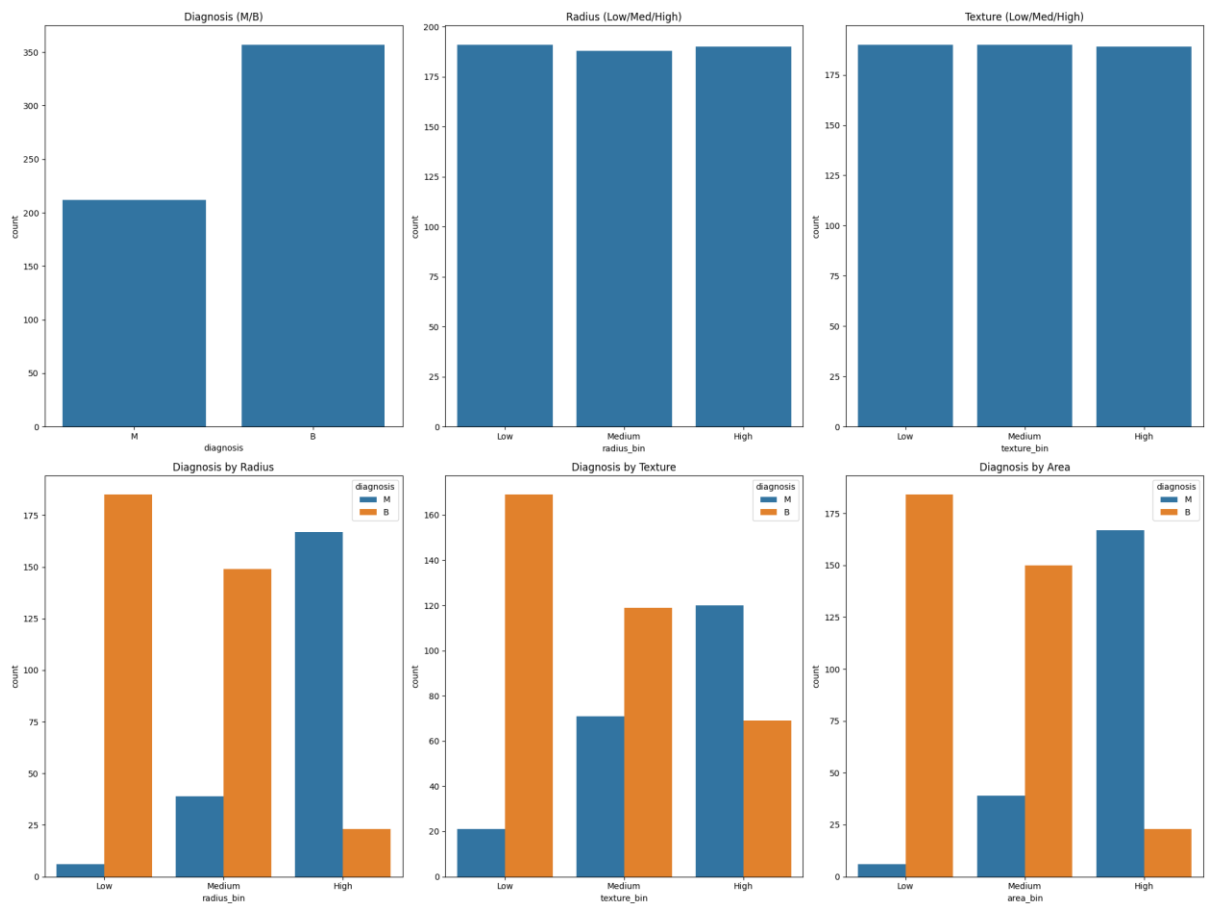
sns.countplot(x='radius_bin', hue='diagnosis', data=cancer_data, ax=axes[3])
axes[3].set_title('Diagnosis by Radius')

sns.countplot(x='texture_bin', hue='diagnosis', data=cancer_data, ax=axes[4])
axes[4].set_title('Diagnosis by Texture')

sns.countplot(x='area_bin', hue='diagnosis', data=cancer_data, ax=axes[5])
axes[5].set_title('Diagnosis by Area')

plt.tight_layout()
plt.show()

```



1.11 Label Encoding (Mengubah Label Kategorikal menjadi Numerik)

Kode tersebut melakukan proses label encoding pada kolom diagnosis, yaitu mengubah nilai kategorikal berupa huruf menjadi nilai numerik agar dapat diproses oleh model machine learning. Pada dataset kanker payudara, nilai diagnosis awalnya ditulis sebagai huruf “M” untuk Malignant (ganas) dan “B” untuk Benign (jinak). Melalui fungsi `.map({'M':1, 'B':0})`, nilai “M” dikonversi menjadi angka 1 dan nilai “B” dikonversi menjadi angka 0. Proses encoding ini sangat penting karena algoritma machine learning, termasuk Gaussian Naive Bayes yang digunakan dalam praktikum ini, hanya menerima input numerik. Selain itu, pengubahan label ke bentuk biner mempermudah perhitungan probabilitas dan evaluasi performa model dalam proses klasifikasi. Dengan adanya encoding tersebut, model dapat membedakan dengan jelas mana sampel yang bersifat jinak dan mana yang bersifat ganas.

```
cancer_data['diagnosis'] = cancer_data['diagnosis'].map({'M':1, 'B':0})

✓ 0.0s

X = cancer_data.drop(columns=['diagnosis', 'radius_bin', 'texture_bin', 'area_bin'], errors='ignore')
y = cancer_data['diagnosis']

✓ 0.0s

Generate + Code + Markdown
Add Markdown Cell

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

✓ 0.0s

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

✓ 0.0s
```

1.12 Pemisahan Fitur dan Label (Feature–Label Splitting)

Kode tersebut digunakan untuk memisahkan dataset menjadi dua bagian utama, yaitu fitur (X) dan label target (y). Pada baris pertama, variabel X dibentuk dengan cara menghapus beberapa kolom tertentu dari DataFrame, yaitu diagnosis, radius_bin, texture_bin, dan area_bin. Kolom diagnosis dihapus karena merupakan label yang ingin diprediksi oleh model, sehingga tidak boleh disertakan sebagai fitur. Sementara itu, kolom radius_bin, texture_bin, dan area_bin merupakan hasil proses binning yang hanya digunakan untuk keperluan visualisasi dan eksplorasi, bukan untuk pelatihan model. Dengan menghapus kolom-kolom tersebut, X hanya berisi fitur numerik asli yang relevan untuk proses klasifikasi. Pada baris kedua, variabel y diisi dengan kolom diagnosis yang telah di-encode sebelumnya, dan menjadi target yang akan diprediksi oleh model machine learning. Pemisahan seperti ini

merupakan langkah fundamental dalam supervised learning karena memisahkan dengan jelas antara input dan output yang digunakan pada proses pelatihan dan pengujian model.

```
cancer_data['diagnosis'] = cancer_data['diagnosis'].map({'M':1, 'B':0})

✓ 0.0s

X = cancer_data.drop(columns=['diagnosis', 'radius_bin', 'texture_bin', 'area_bin'], errors='ignore')
y = cancer_data['diagnosis']

✓ 0.0s

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

✓ 0.0s

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

✓ 0.0s
```

1.13 Pembagian Data (Train-Test Split) dengan Stratifikasi

Kode tersebut digunakan untuk membagi dataset menjadi dua bagian utama, yaitu data latih (training set) dan data uji (testing set). Pembagian ini dilakukan menggunakan fungsi `train_test_split()` dengan parameter `test_size=0.2`, yang berarti 20% data akan digunakan sebagai data uji, sementara sisanya yaitu 80% digunakan sebagai data latih. Tujuan dari pembagian ini adalah agar model dapat belajar dari sebagian besar data yang tersedia, kemudian diuji menggunakan data yang belum pernah dilihat sebelumnya untuk mengukur kemampuan generalisasinya. Parameter `random_state=42` memastikan bahwa proses pembagian data bersifat konsisten dan dapat direproduksi kapan pun kode dijalankan. Selain itu, parameter `stratify=y` digunakan untuk menjaga proporsi kelas pada data latih dan data uji tetap sama seperti pada dataset asli. Dengan adanya stratifikasi ini, jumlah kasus kanker jinak dan ganas pada kedua set data tetap seimbang, sehingga mencegah model bias

terhadap salah satu kelas. Secara keseluruhan, proses ini penting untuk memastikan bahwa model dilatih dan diuji secara adil dan representatif.

```
cancer_data['diagnosis'] = cancer_data['diagnosis'].map({'M':1, 'B':0})

✓ 0.0s

X = cancer_data.drop(columns=['diagnosis', 'radius_bin', 'texture_bin', 'area_bin'], errors='ignore')
y = cancer_data['diagnosis']

✓ 0.0s

Generate + Code + Markdown
Add Markdown Cell

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

✓ 0.0s

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

✓ 0.0s
```

1.14 Standardisasi / Normalisasi Fitur (Feature Scaling)

Kode tersebut digunakan untuk melakukan proses standardisasi fitur, yaitu langkah penting dalam machine learning untuk menyamakan skala seluruh variabel numerik. Pada baris pertama, objek `StandardScaler()` dibuat untuk melakukan transformasi data menjadi distribusi dengan mean = 0 dan standard deviation = 1. Hal ini penting karena setiap fitur dalam dataset kanker payudara memiliki rentang nilai yang berbeda-beda, misalnya fitur `area_mean` bisa memiliki nilai ratusan hingga ribuan, sedangkan fitur `smoothness_mean` hanya bernilai sekitar 0.1. Jika skala tidak diseragamkan, model seperti Gaussian Naive Bayes dapat menjadi kurang akurat karena fitur yang memiliki skala besar akan mendominasi perhitungan probabilitas. Pada baris kedua, dilakukan `fit_transform()` pada data training untuk menghitung nilai mean dan standar deviasi dari fitur, lalu menerapkan standardisasi pada data tersebut. Pada baris ketiga, proses `transform()` diterapkan pada data testing menggunakan parameter yang telah dihitung dari data training, sehingga

mencegah kebocoran informasi (data leakage). Dengan standardisasi ini, model dapat belajar secara lebih efektif dan menghasilkan performa prediksi yang lebih stabil.

```
cancer_data['diagnosis'] = cancer_data['diagnosis'].map({'M':1, 'B':0})

✓ 0.0s

X = cancer_data.drop(columns=['diagnosis', 'radius_bin', 'texture_bin', 'area_bin'], errors='ignore')
y = cancer_data['diagnosis']

✓ 0.0s

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

✓ 0.0s

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

✓ 0.0s
```

1.15 Inisialisasi dan Pelatihan Model Naive Bayes

Kode tersebut digunakan untuk membuat dan melatih model klasifikasi menggunakan algoritma Gaussian Naive Bayes. Pada baris pertama, dilakukan import kelas GaussianNB dari modul sklearn.naive_bayes, yang merupakan algoritma Naive Bayes yang dirancang untuk data dengan fitur numerik dan diasumsikan mengikuti distribusi Gaussian atau normal. Pada baris kedua, objek model dibuat dengan memanggil GaussianNB(), sehingga variabel nb_model kini berisi model kosong yang siap dilatih. Pada baris ketiga, model tersebut dilatih menggunakan data training yang telah distandarisasi, yaitu X_train_scaled sebagai fitur dan y_train sebagai label diagnosis. Melalui proses fit(), model mempelajari pola distribusi fitur dari masing-masing kelas (benign dan malignant), kemudian menghitung probabilitas yang digunakan untuk melakukan prediksi di tahap selanjutnya. Proses pelatihan ini merupakan inti dari machine learning, di mana model mempelajari perbedaan

karakteristik antara sampel jinak dan ganas sehingga mampu mengklasifikasikan data baru secara akurat.

```
from sklearn.naive_bayes import GaussianNB
nb_model = GaussianNB()
nb_model.fit(X_train_scaled, y_train)
```

✓ 0.0s

▼ GaussianNB ⓘ ?

► Parameters

1.16 Evaluasi Akurasi Model (Training & Testing Accuracy)

Kode di atas digunakan untuk mengevaluasi performa model Gaussian Naive Bayes dengan mengukur tingkat akurasi pada data training dan data testing. Pada baris pertama, perintah `nb_model.score(X_train_scaled, y_train)` menghitung akurasi model terhadap data yang digunakan untuk melatih model, yaitu training set. Nilai ini menunjukkan seberapa baik model mampu mengenali pola-pola yang telah dipelajarinya. Pada baris kedua, `nb_model.score(X_test_scaled, y_test)` menghitung akurasi pada testing set, yaitu data baru yang tidak pernah dilihat model sebelumnya. Akurasi testing memberikan gambaran mengenai kemampuan model dalam melakukan generalisasi terhadap data nyata. Jika akurasi training dan testing keduanya tinggi dan nilainya tidak terlalu jauh, maka model dianggap stabil dan tidak mengalami overfitting. Baris selanjutnya menampilkan hasil akurasi tersebut ke layar sehingga pengguna dapat langsung membandingkan performa model pada kedua jenis data. Tahap evaluasi ini sangat penting untuk memastikan bahwa model memberikan prediksi yang andal dan akurat, terutama dalam konteks medis seperti diagnosis kanker payudara.

```
train_acc = nb_model.score(X_train_scaled, y_train)
test_acc = nb_model.score(X_test_scaled, y_test)

print("Training Accuracy:", train_acc)
print("Testing Accuracy:", test_acc)
```

✓ 0.0s

Training Accuracy: 0.945054945054945
Testing Accuracy: 0.9210526315789473

1.17 Evaluasi Model Menggunakan Confusion Matrix (Confusion Matrix Visualization)

Kode tersebut digunakan untuk mengevaluasi performa model Gaussian Naive Bayes melalui pembuatan confusion matrix. Pada baris pertama, model melakukan prediksi terhadap data testing yang telah distandarisasi menggunakan perintah `nb_model.predict(X_test_scaled)`. Hasil prediksi tersebut disimpan ke dalam variabel `y_pred`. Selanjutnya, fungsi `confusion_matrix(y_test, y_pred)` digunakan untuk menghitung jumlah prediksi benar dan salah berdasarkan kelas sebenarnya dan kelas yang diprediksi, sehingga diperoleh empat nilai penting: True Positive (TP), True Negative (TN), False Positive (FP), dan False Negative (FN). Confusion matrix ini kemudian divisualisasikan dalam bentuk heatmap menggunakan Seaborn melalui `sns.heatmap()`, dengan anotasi angka untuk memudahkan pembacaan. Grafik diberi judul “Confusion Matrix” serta label pada sumbu X dan Y untuk menunjukkan hubungan antara nilai aktual dan nilai prediksi. Visualisasi ini sangat bermanfaat untuk melihat performa model secara lebih detail, terutama dalam konteks klasifikasi medis, karena menunjukkan tidak hanya akurasi total tetapi juga bagaimana model menangani masing-masing kelas (ganas vs jinak). Dengan melihat heatmap, pengguna dapat dengan mudah mengetahui apakah model lebih sering melakukan kesalahan pada prediksi kanker ganas atau jinak.



1.18 Evaluasi Model Menggunakan Classification Report

Kode `print(classification_report(y_test, y_pred))` digunakan untuk menampilkan classification report, yaitu ringkasan metrik evaluasi yang lebih mendetail dibandingkan akurasi dan confusion matrix. Laporan ini mencakup nilai precision, recall, f1-score, dan support untuk masing-masing kelas diagnosis, yaitu 0 (benign/jinak) dan 1 (malignant/ganas). Precision menunjukkan tingkat ketepatan prediksi positif model, terutama penting untuk menghindari kesalahan diagnosis pada kasus kanker ganas. Recall menunjukkan kemampuan model dalam mengenali seluruh sampel positif, sehingga sangat penting dalam konteks medis agar tidak ada kasus kanker ganas yang terlewat. F1-score merupakan kombinasi dari precision dan recall yang memberikan gambaran keseimbangan performa model terhadap kedua metrik tersebut. Sementara itu, support menunjukkan jumlah sampel aktual pada masing-masing kelas dalam data uji. Dengan melihat classification report, kita dapat menilai performa model dengan lebih

komprehensif dan mengetahui apakah model bekerja merata pada kedua kelas atau cenderung bias terhadap kelas tertentu. Evaluasi ini sangat penting karena pada kasus diagnosis kanker, kesalahan klasifikasi dapat berdampak signifikan terhadap keputusan medis.

```
print(classification_report(y_test, y_pred))
```

11] ✓ 0.0s

	precision	recall	f1-score	support
0	0.92	0.96	0.94	72
1	0.92	0.86	0.89	42
accuracy			0.92	114
macro avg	0.92	0.91	0.91	114
weighted avg	0.92	0.92	0.92	114

1.19 Cross Validation (5-Fold Cross Validation Accuracy)

Kode tersebut digunakan untuk melakukan evaluasi performa model Gaussian Naive Bayes menggunakan metode 5-Fold Cross Validation. Pada baris pertama, fungsi `cross_val_score()` dipanggil dengan model `nb_model`, fitur `X`, dan label `y`, serta parameter `cv=5` yang berarti dataset dibagi menjadi 5 bagian atau fold. Dalam proses ini, model akan dilatih dan diuji sebanyak lima kali menggunakan kombinasi data yang berbeda, sehingga setiap fold akan bergantian menjadi data uji sementara empat fold lainnya digunakan untuk pelatihan. Parameter `scoring='accuracy'` menunjukkan bahwa metrik evaluasi yang digunakan adalah akurasi. Hasil akurasi dari setiap fold disimpan dalam variabel `cv_nb`. Baris selanjutnya mencetak nilai akurasi dari setiap fold, kemudian menghitung dan menampilkan nilai rata-rata akurasi serta standar deviasi. Rata-rata akurasi yang diperoleh sebesar 0.9385, atau sekitar 93.85%, menunjukkan bahwa model bekerja dengan sangat baik. Sementara itu, nilai standar deviasi yang kecil (0.0145) menandakan bahwa performa model stabil dan konsisten di berbagai pembagian data. Evaluasi cross validation ini sangat penting karena memberikan gambaran menyeluruh mengenai keandalan model, tidak hanya berdasarkan satu kali pembagian data tetapi dari berbagai skenario pelatihan dan pengujian.

```
from sklearn.model_selection import cross_val_score
cv_nb = cross_val_score(nb_model, X, y, cv=5, scoring='accuracy')
print("\nNaive Bayes Cross Validation Accuracy (5-Fold):")
print("Scores:", cv_nb)
print("Mean Accuracy:", cv_nb.mean())
print("Std Deviation:", cv_nb.std())
```

✓ 0.0s

Naive Bayes Cross Validation Accuracy (5-Fold):
Scores: [0.92105263 0.92105263 0.94736842 0.94736842 0.95575221]
Mean Accuracy: 0.9385188635305075
Std Deviation: 0.014585994424363306

Kesimpulan:

Berdasarkan seluruh rangkaian proses analisis dan pemodelan yang telah dilakukan menggunakan algoritma Gaussian Naive Bayes, dapat disimpulkan bahwa model mampu memberikan performa klasifikasi yang sangat baik pada dataset kanker payudara (Breast Cancer Wisconsin Dataset). Setelah dilakukan preprocessing berupa penghapusan kolom tidak relevan, encoding label diagnosis, pembagian data 80% untuk training dan 20% untuk testing, serta standarisasi fitur, model menunjukkan akurasi yang tinggi baik pada data latih maupun data uji. Evaluasi lebih lanjut menggunakan confusion matrix* dan classification report menunjukkan bahwa model memiliki kemampuan yang baik dalam membedakan antara kanker jinak dan ganas, dengan nilai precision, recall, dan f1-score yang seimbang. Selain itu, hasil 5-Fold Cross Validation menghasilkan rata-rata akurasi sebesar 93.85% dengan standar deviasi rendah, menandakan bahwa model stabil dan konsisten pada berbagai pembagian data. Secara keseluruhan, dapat disimpulkan bahwa Gaussian Naive Bayes merupakan metode klasifikasi yang efektif dan handal untuk mendeteksi jenis kanker payudara berdasarkan fitur fisiologis sel, sehingga dapat digunakan sebagai alat bantu awal dalam proses diagnosis medis.

Link:

- <https://www.kaggle.com/datasets/prince7489/fitness-tracker-dataset>
- https://github.com/orzaynd/machine_learning/tree/master/praktikum/praktikum7
- https://github.com/orzaynd/machine_learning/tree/master/tugas